

E4-TP1 : INSTALLATION D'UNE PLATEFORME DE DÉVELOPPEMENT IOT

Prérequis : n/a	Matériels et documentation : - Visual Studio Code - Extensions : PlatformIO et C++ - Carte nodeMCU ESP32	Durée : 1H00
--	---	-----------------------------------

Problématique	Comment installer une plateforme de développement IoT sous GNU/Linux ?
Objectifs	À la fin de la séance, vous devrez être capable de : - Installer un environnement de développement IoT - Mettre en œuvre une carte de développement basée sur le microcontrôleur ESP32 - Écrire un programme simple en C++ et le téléverser dans une carte de développement

On vous demande d'installer et de tester une plateforme de développement IoT comprenant l'installation d'un environnement de développement et la mise en œuvre d'une carte de prototypage nodeMCU basée sur le microcontrôleur ESP32.

1 PRÉSENTATION DE L'ESP32 NODEMCU



Le module nodeMCU ESP32 est un dispositif de développement basé sur le microcontrôleur ESP32 de Espressif Systems.

Il offre une plateforme de prototypage rapide et abordable pour les projets **IoT (Internet of Things)** et les applications embarquées.

L'ESP32 est un **SoC (System-on-Chip)**, en effet, il intègre plusieurs composants essentiels d'un système sur une seule puce : microcontrôleur, connectivité Wi-Fi et Bluetooth, prise en charge de bus informatiques...

Microcontrôleur ESP32 : Le cœur du nodeMCU est le microcontrôleur ESP32, qui est un système à puce à double cœur Tensilica Xtensa LX6 avec connectivité Wi-Fi et Bluetooth intégrée.

Wi-Fi et Bluetooth : Le module nodeMCU prend en charge les réseaux Wi-Fi 802.11 b/g/n et intègre un Bluetooth classique et BLE (*Bluetooth Low Energy*).

Interfaces GPIO : Le module nodeMCU dispose de plusieurs broches **GPIO (General Purpose Input/Output)** qui permettent la connexion à divers capteurs, actionneurs et périphériques externes. Ces broches prennent en charge des interfaces telles que UART, SPI et I2C.

USB-TTL : Le module nodeMCU est équipé d'un convertisseur USB-TTL qui facilite la programmation et la communication série avec un ordinateur hôte.



Photo 1 : NodeMCU ESP32

CPU ESP-WROOM-32 (Tensilica Xtensa LX6)
Voltage : 3.3V
Flash : 4000 kB
RAM : 520 kB
EEPROM : 448 kB
Clock speed : 240MHz
WiFi : Yes (802.11 b/g/n)
Bluetooth : Yes BLE : Yes

Tableau 1 : Caractéristiques essentielles de la carte nodeMCU ESP32

Alimentation : Le module nodeMCU peut être alimenté via une source USB ou une alimentation externe, offrant ainsi une grande flexibilité dans les scénarios d'utilisation.

2 ACTIVITÉ 1 : INSTALLATION DE VISUAL STUDIO CODE



Visual Studio Code est un éditeur de code extensible développé par Microsoft pour Windows, Linux et macOS. Les fonctionnalités incluent la prise en charge du débogage, la mise en évidence de la syntaxe, la complétion intelligente du code et l'outil de versioning Git.

Nous utiliserons le langage de haut niveau **C/C++** pour programmer les microcontrôleurs ESP32, il faudra donc installer l'**extension C/C++** de Microsoft.



De plus, pour obtenir la chaîne de compilation croisée, la prise en charge de la carte et les bibliothèques, nous utiliserons l'**écosystème PlatformIO** sous la forme d'une extension pour VS Code. Cette extension proposera l'utilisation des deux Framework pour les ESP32 : Arduino Core ou ESP-IDF.

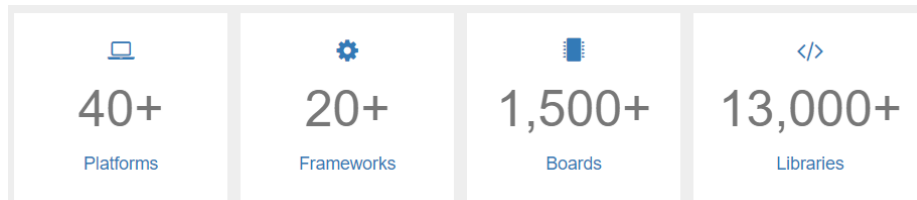


Figure 1 : PlatformIO en quelques chiffres

Vous allez dans cette activité installer Visual Studio Code et les deux extensions nécessaires pour programmer les cartes à base de ESP32.

- (01) Télécharger le paquet **.deb** de **VS Code** à partir du site : code.visualstudio.com. Installer le paquet télécharger en ligne de commande.
- (02) Installer en ligne de commande les paquets : **python3-venv** et **python3-virtualenv**.
- (03) Ouvrir VS Code et choisir l'onglet Extension. Rechercher et installer les extensions : **C/C++** de Microsoft et **PlatformIO IDE**.
- (04) Choisir l'extension PlatformIO et créer un projet nommé **projet_1_esp32** en sélectionnant la carte : **Espressif ESP32 Dev Module** et le Framework : **Arduino**.



3 ACTIVITÉ 2 : ACCÈS AUX PORTS SÉRIES DU PC



Une fois que le code C++ sera écrit dans l'éditeur, il faudra le **compiler** pour obtenir un binaire puis **téléverser** ce binaire dans la carte en utilisant un câble USB. Il faudra donc créer des règles d'accès aux périphériques séries sur votre distribution Linux pour l'extension PlatformIO. De plus, vous devrez ajouter les groupes à votre utilisateur, groupes liés aux ports séries et aux points de montage.

- (05) En utilisant la commande **wget**, télécharger le fichier **99-platformio-udev.rules** dont le l'URL est donnée ci-dessous. Déplacer en ligne de commande le fichier dans le répertoire **/etc/udev/rules.d/**. Redémarrer en ligne de commande le service **udev**.

<https://raw.githubusercontent.com/platformio/platformio-core/develop/platformio/assets/system/99-platformio-udev.rules>

- (06) Ajouter à votre utilisateur les groupes **dialout** et **plugdev** avec la commande **usermod**.

4 PRÉSENTATION RAPIDE DE PLATFORMIO

La page d'accueil de PlatformIO permet de configurer les projets. Vous trouverez des outils permettant d'inspecter un projet, de gérer les bibliothèques, les cartes et les plateformes supportées avec les Framework installés.

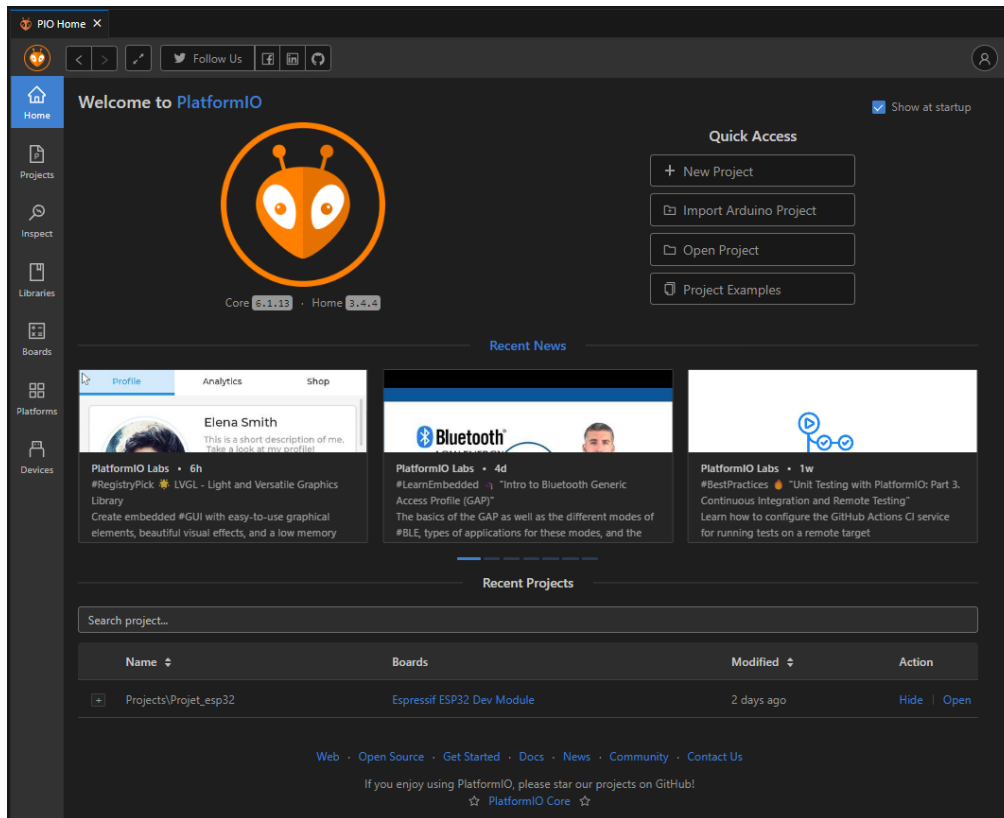


Figure 2 : Capture d'écran de la page d'accueil de PlatformIO

Un projet suit l'architecture ci-contre. Les codes sources de l'application seront situés dans le dossier **src**. Par défaut, pour un Framework Arduino vous trouverez le fichier **main.cpp** dans ce dossier.

Le fichier **platformio.ini** contiendra la configuration du projet. Les options de configuration sont disponibles dans la documentation docs.platformio.org/page/projectconf.html.

Pour compiler le projet, téléverser le binaire et suivre une communication, vous utiliserez une des 3 icônes suivante :

-  **Fabriquer (build)** : compilation des sources
-  **Téléverser (upload)** : programmation du microcontrôleur avec le binaire généré
-  **Ouvrir le moniteur série (serial monitor)**

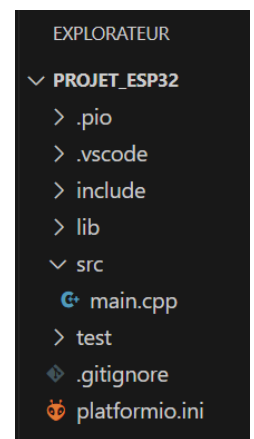


Figure 3 : Architecture de PlatformIO

5 ACTIVITÉ 3 : TEST DE LA PLATEFORME DE DÉVELOPPEMENT

Dans cette activité, vous allez tester votre installation en écrivant un code simple, le compiler et le téléverser dans la carte.

- (07) Relever sur la Figure 4 : Schéma structurel de la carte nodeMCU ESP32, le numéro du GPIO du microprocesseur ESP32 sur lequel est connecté la LED bleue.
- (08) Ouvrir le fichier **main.cpp** et écrire le code suivant permettant de faire clignoter la LED bleue de la carte. Vous remplacerez le **x** par le numéro du GPIO trouvé précédemment.

```
/*  
  Blink  
  Blinking blue LED on card  
  ESP32 Dev Module  
*/  
  
#include <Arduino.h>  
  
const int8_t LED = GPIO_NUM_x;  
  
void setup() {  
  pinMode(LED, OUTPUT);  
}  
  
void loop() {  
  digitalWrite(LED, HIGH);  
  delay(500);  
  digitalWrite(LED, LOW);  
  delay(500);  
}
```



- (09) Brancher la carte nodeMCU ESP32 à votre machine avec le câble USB. Compiler et téléverser le programme dans la carte. Vérifier le fonctionnement de la carte.

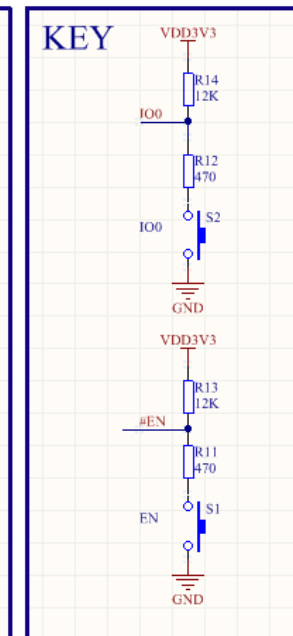
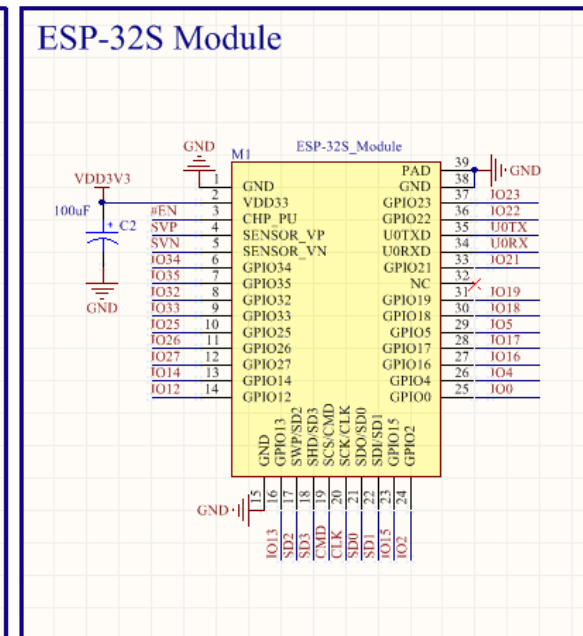
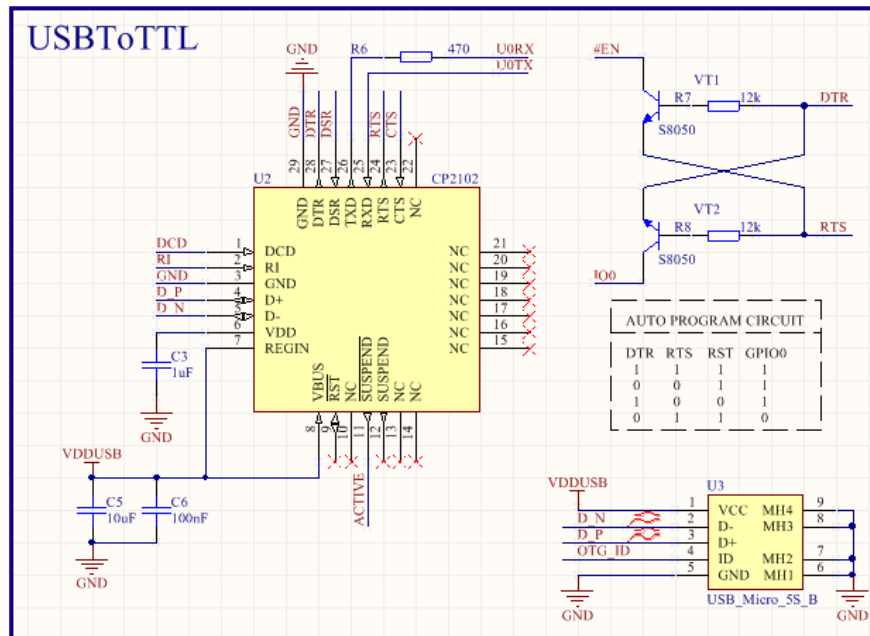
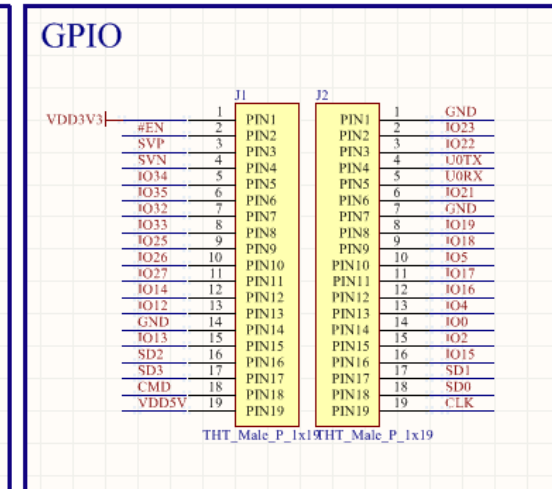
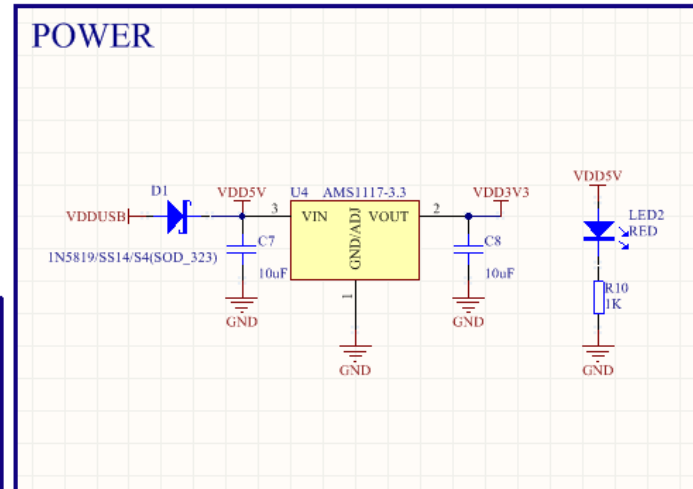
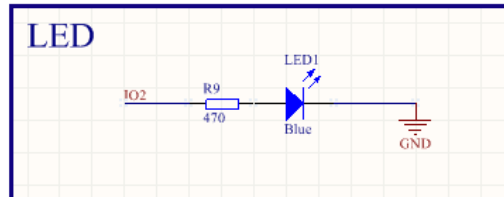


Figure 4 : Schéma structurel de la carte nodeMCU ESP32